

Excel Bulk Upload Backend (Express + Sequelize + PostgreSQL)

This document contains a complete working backend you can drop into a `backend/` folder. It implements the `/bulkUpload` endpoint that matches the React client created earlier.

Folder structure

```
backend/
├── config/
│   └── db.js
├── controllers/
│   └── bulkController.js
├── models/
│   └── User.js
├── routes/
│   └── bulkRoutes.js
├── .env.example
├── app.js
├── package.json
└── README.md
```

1) package.json

```
{
  "name": "excel-bulk-upload-server",
  "version": "1.0.0",
  "type": "module",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.0.0",
    "express": "^4.18.2",
    "pg": "^8.10.0",
    "sequelize": "^6.32.1"
  },
  "devDependencies": {
    "nodemon": "^3.0.0"
  }
}
```

```
}  
}
```

2) .env.example

```
PORT=5000  
DB_NAME=your_database  
DB_USER=postgres  
DB_PASS=your_password  
DB_HOST=localhost  
DB_PORT=5432
```

Copy this to `.env` and fill values.

3) config/db.js

```
import { Sequelize } from "sequelize";  
import dotenv from "dotenv";  
dotenv.config();  
  
const sequelize = new Sequelize(  
  process.env.DB_NAME,  
  process.env.DB_USER,  
  process.env.DB_PASS,  
  {  
    host: process.env.DB_HOST || "localhost",  
    port: process.env.DB_PORT ? Number(process.env.DB_PORT) : 5432,  
    dialect: "postgres",  
    logging: false,  
    pool: {  
      max: 10,  
      min: 0,  
      acquire: 30000,  
      idle: 10000,  
    },  
  },  
);  
  
export default sequelize;
```

4) models/User.js

```
import { DataTypes } from "sequelize";
import sequelize from "../config/db.js";

const User = sequelize.define(
  "User",
  {
    id: {
      type: DataTypes.INTEGER,
      autoIncrement: true,
      primaryKey: true,
    },
    name: {
      type: DataTypes.STRING,
      allowNull: false,
    },
    email: {
      type: DataTypes.STRING,
      allowNull: false,
      validate: { isEmail: true },
    },
    phone: {
      type: DataTypes.STRING,
      allowNull: true,
    },
    age: {
      type: DataTypes.INTEGER,
      allowNull: true,
    },
  },
  {
    tableName: "users",
    timestamps: true,
  }
);

export default User;
```

5) controllers/bulkController.js

```
import User from "../models/User.js";

// SERVER-SIDE VALIDATION (should mirror frontend)
function validateRow(row) {
  const errors = [];
```

```

const name = (row.Name ?? row.name ?? "").toString().trim();
const email = (row.Email ?? row.email ?? "").toString().trim();
const phone = (row.Phone ?? row.phone ?? "").toString().trim();
const ageRaw = row.Age ?? row.age ?? null;

const age = ageRaw === null || ageRaw === "" ? null : Number(ageRaw);

if (!name) errors.push("Name is required");
if (!email) errors.push("Email is required");
else {
  if (!/^[\s@]+\s+@[\s@]+\s+[\s@]+$/i.test(email)) errors.push("Invalid
email");
}

if (phone) {
  if (!/^([0-9+\\-\\s()]*$)/i.test(phone)) errors.push("Invalid phone format");
  if (phone.replace(/D/g, "").length < 7) errors.push("Phone looks too
short");
}

if (age !== null) {
  if (!Number.isInteger(age)) errors.push("Age must be integer");
  if (age < 0 || age > 150) errors.push("Age out of range");
}

return errors;
}

export const bulkUpload = async (req, res) => {
  const { rows, meta } = req.body || {};

  if (!rows || !Array.isArray(rows)) {
    return res.status(400).json({ message: "Rows missing or invalid" });
  }

  const failed_rows = [];
  const validRows = [];

  rows.forEach((row) => {
    const errors = validateRow(row);
    if (errors.length > 0) failed_rows.push({ row, errors });
    else validRows.push(row);
  });

  if (validRows.length === 0) {
    return res.json({
      file: meta?.fileName ?? null,
      total_received: rows.length,
      inserted: 0,
      inserted_ids: [],
      failed_count: failed_rows.length,
    });
  }

```

```

        failed_rows,
    });
}

try {
    const formattedRows = validRows.map((r) => ({
        name: r.Name ?? r.name,
        email: r.Email ?? r.email,
        phone: r.Phone ?? r.phone ?? null,
        age: r.Age ?? r.age ?? null,
    }));

    // Bulk create - returns instances when `returning: true` supported by
    dialect
    const inserted = await User.bulkCreate(formattedRows, { returning:
    true });

    return res.json({
        file: meta?.fileName ?? null,
        total_received: rows.length,
        inserted: inserted.length,
        inserted_ids: inserted.map((i) => i.id),
        failed_count: failed_rows.length,
        failed_rows,
    });
} catch (err) {
    console.error("Bulk insert error:", err);
    return res.status(500).json({ message: "Bulk upload failed", error:
    err.message });
}
};

```

6) routes/bulkRoutes.js

```

import express from "express";
import { bulkUpload } from "../controllers/bulkController.js";

const router = express.Router();

router.post("/bulkUpload", bulkUpload);

export default router;

```

7) app.js

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import sequelize from "../config/db.js";
import bulkRoutes from "../routes/bulkRoutes.js";
import User from "../models/User.js"; // import models so sequelize knows them

dotenv.config();

const app = express();
app.use(cors());
app.use(express.json({ limit: "50mb" }));

// routes
app.use("/", bulkRoutes);

// health
app.get("/health", (req, res) => res.json({ ok: true, env:
process.env.NODE_ENV || "development" }));

const PORT = process.env.PORT || 5000;

async function start() {
  try {
    await sequelize.authenticate();
    console.log("DB connection OK");

    // Sync models (safe in dev). In production use migrations.
    await sequelize.sync({ alter: true });
    console.log("DB synced");

    app.listen(PORT, () => console.log(`Server running on http://localhost:${
PORT}`));
  } catch (err) {
    console.error("Startup error:", err);
    process.exit(1);
  }
}

start();
```

8) README.md (quick setup)

```
# Excel Bulk Upload Backend
```

1. Copy ``.env.example`` to ``.env`` and set DB values.
2. ``npm install``
3. Create the Postgres database matching DB_NAME in ``.env``.
4. ``npm run dev`` to start with nodemon or ``npm start``.

Endpoint: POST `http://localhost:5000/bulkUpload`

Payload: `{ meta: {...}, rows: [ { Name, Email, Phone, Age, ... }, ... ] }`

Response example:

```
{
  "file": "sample.xlsx",
  "total_received": 10,
  "inserted": 8,
  "inserted_ids": [1,2,3,4,5,6,7,8],
  "failed_count": 2,
  "failed_rows": [ { row: {...}, errors: [..] }, ... ]
}
```

Notes & next steps

- This implementation uses `sequelize.sync({ alter: true })` which will create/alter the `users` table automatically. For production use, replace with proper migrations.
- For very large Excel files (>10k rows) you may want a chunked upload strategy or background job processing. If you want that, I can add chunking + resumable upload + status endpoint.
- If you prefer raw `pg` instead of Sequelize, I can convert the controller to use parameterized `INSERT` with transaction and `COPY`.

If you want, I can now:

- Generate these files in the canvas as separate code files (one per file) so you can copy them faster.
- Convert to a raw `pg` implementation (no ORM) for slightly better bulk performance.
- Add chunked upload + progress using SSE or websockets.

Tell me which next step you want and I'll produce it immediately.